

AutoKart Firmware Bench Report

Changes, Errors, and Pinout Comparison

Scope. This compact report records the firmware and bench wiring developed in the AutoKart-firmware repository for ESP32-S3 BLDC testing, steering/brake tests, and RA-02 LoRa emergency-stop experiments. It is a bench record, not a road ready safety certification.

1. Current project state

BLDC controller	Working bench test. The current branch uses bench/full_bldc_test.py, GP8630N output, and the web interface. The non-LoRa controller is the default stable bench file.
Throttle/output	The test moved from approximately 3 V to a 5 V maximum. The current file reports MAX_VOLTAGE = 5.00 V.
Steering and brake	Separate open-loop TB6600 test in bench/steering_brake_test.py. Software limits are steering -45 to +45 degrees and brake 0 to 30 degrees.
LoRa E-stop	Radio-only Pico/ESP32 tests worked. The combined LoRa/BLDC file remains separate for continued integration and safety testing.

2. Changes made

- Repository organized into bench/, firmware/, docs/, and images/.
- BLDC transistor mapping changed to BRK GPIO13, EN GPIO14, and F/R GPIO15. The F/R pin stayed on GPIO15; BRK and EN were exchanged.
- Throttle was extended from a 3 V test range to a 5 V maximum. The UI slider was changed to 0.01 V resolution and its request handling was made smoother by sending the latest pending value instead of flooding the web server.
- CW and CCW output were compared. The large directional difference was treated as an engineering issue requiring measurement, not as proof that target SV alone was wrong.
- The ESP32-S3 RA-02 test moved to GPIO4/5/6 for SPI and GPIO7 for CS, leaving GPIO13/14/15 free for BLDC controls.
- Pico transmitter messages are ESTOP,0,sequence at startup/release and ESTOP,1,sequence while the LAY37 is pressed/open. The stable non-LoRa BLDC file was restored as the default; LoRa integration stays separate.
- Steering and brake pulse outputs were separated onto GPIO4-6 and GPIO10-12. Steering is represented as a 90 degree software window centered at zero.

3. BLDC pinout comparison

Signal	Earlier	Current
EN	GPIO13	GPIO14
BRK	GPIO14	GPIO13
F/R	GPIO15	GPIO15
GP8630N SDA	GPIO8	GPIO8
GP8630N SCL	GPIO9	GPIO9
GP8630N address	0x58	0x58
SV wiring	OUT -> SV	OUT -> SV; GND -> COM

BRK and EN were exchanged; F/R stayed on GPIO15. The GP8630N remains on I2C GPIO8/9 at address 0x58.

4. RA-02 pinout comparison

RA-02	Old S3	Combined S3	Pico 2
3.3V/GND	3V3/GND	3V3/GND	3V3/GND
NSS/CS	GPIO16	GPIO7	GP17
SCK	GPIO14	GPIO4	GP18
MOSI	GPIO13	GPIO5	GP19
MISO	GPIO15	GPIO6	GP16
RESET	GPIO10	GPIO10	GP20
DIO0	GPIO18	GPIO18	GP21

The alternate ESP32-S3 radio map prevents SPI from sharing BLDC transistor pins 13, 14, and 15. RA-02 wiring is 3.3 V only, with common ground and local decoupling recommended.

5. Steering, brake, and E-stop pinout

Function	Pin	Connection / state
Steer STEP	GPIO4	TB6600 PUL+
Steer DIR	GPIO5	TB6600 DIR+
Steer ENABLE	GPIO6	TB6600 ENA+; active-low
Brake STEP	GPIO10	TB6600 PUL+
Brake DIR	GPIO11	TB6600 DIR+
Brake ENABLE	GPIO12	TB6600 ENA+; active-low
LAY37 input	Pico GP22	NC -> GP22; COM -> GND; 4.7k -> 3.3V
LAY37 healthy	GP22 LOW	NC closed
LAY37 stop	GP22 HIGH	NC open / pressed / wire fault

The steering test is open-loop. It assumes the mechanism starts physically centered and needs the real gear ratio calibrated before the displayed angle can be trusted.

6. Errors and issues encountered

- **Serial port busy:** mpremote failed when Thonny, a REPL, or another Python process owned the device. The remedy is one serial client at a time, verified with lsof or fuser.
- **Raw REPL failure:** could not enter raw repl and b'' occurred when the board was busy, the port was locked, or the wrong board/image state was selected. Reset and reconnect after stopping the running program.
- **Port confusion:** ACM0, ACM1, and ttyUSB0 are dynamic names. RP2 bootloader mode appears as mass storage, not necessarily as a serial port. Rediscover the port after each reconnect.
- **Missing network module:** ImportError: no module named 'network' indicated ESP32-specific code running on an incompatible board/image or on the wrong selected device.
- **RA-02 packets initially not seen:** pin mapping, one-shot timing, receiver state, and serial selection were being tested together. Standalone transmitter/receiver tests were used to isolate the radio link.
- **Integrated LoRa error:** the combined controller stopped with name 'reason' isn't defined. The stable non-LoRa BLDC controller was kept separate while the integrated file remains under repair and retest.
- **Slow-speed motor stop:** pin changes, throttle ramp/watchdog behavior, and drive response were being changed in the same sequence. Stabilize pin mapping and test voltage, ramp, direction, and load independently.
- **CW/CCW speed mismatch:** possible causes include motor/controller wiring, mechanical load, F/R logic, brake release, or drive-stage asymmetry. Measure actual SV in both directions before assigning the cause to target SV.

7. Validation summary and next step

Test	Result
ESP32-S3 BLDC bench Pico 2 + RA-02	User confirmed stable after fix Button and ESTOP,1 packets observed
ESP32-S3 RA-02	Standalone test; integrate next
Steering angle	-45/+45 limits; calibrate gear ratio

Recommended next test. keep bench/full_bldc_test.py as the controller, verify BRK/EN/F-R at zero SV, then run the Pico transmitter and ESP32 receiver alone. Confirm ESTOP,0 at startup/release and ESTOP,1 when pressed. Only then combine radio handling with the BLDC loop, with wheels lifted and a physical power cut available.

Sources: bench/full_bldc_test.py, bench/full_bldc_lora_estop_esp32s3.py, bench/pico2_ra02_estop_transmitter.py, bench/steering_brake_test.py, and docs/ wiring notes. Repository history reviewed through commit 4abfe30.